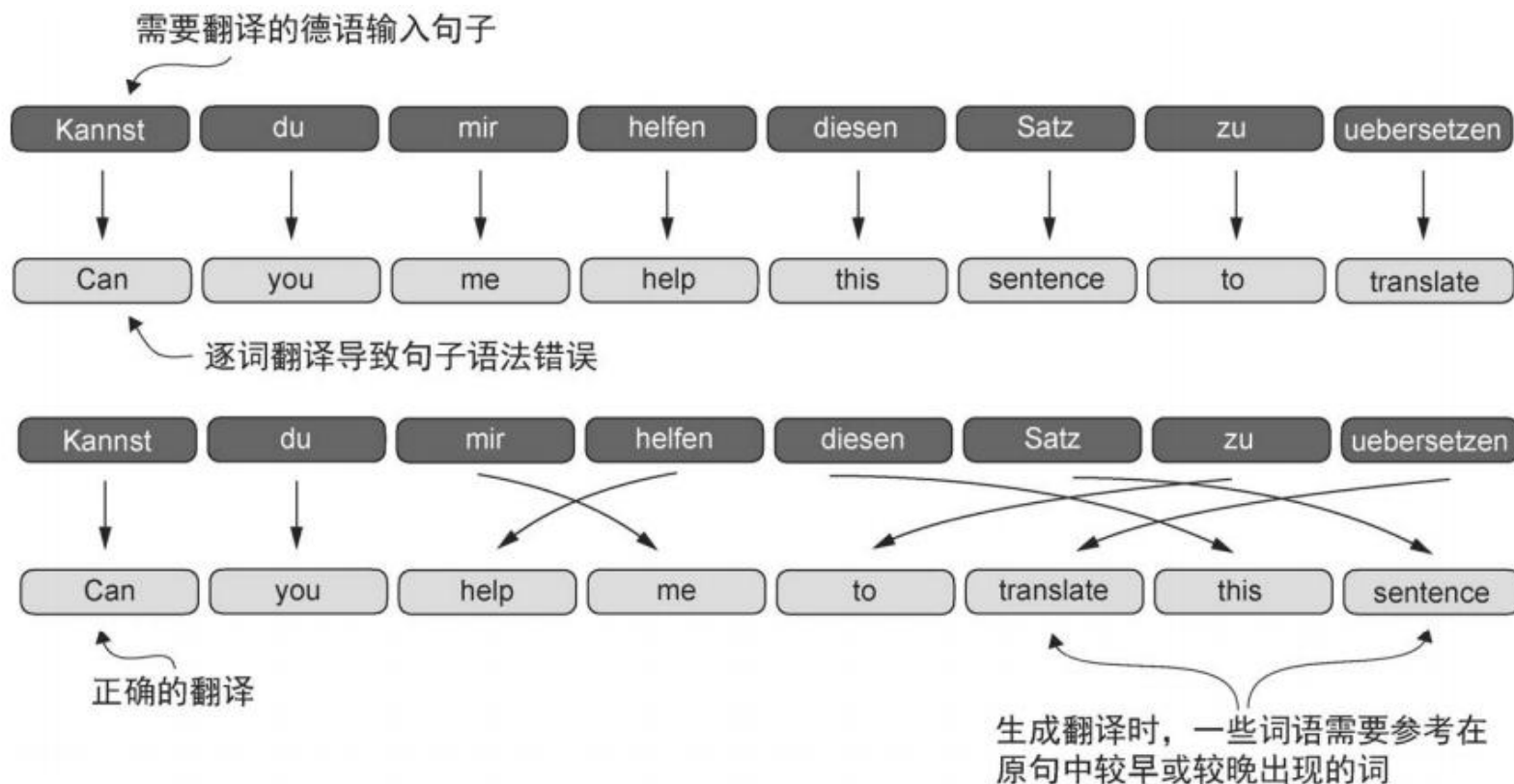




第3章： 注意力机制

背景：传统框架下的“记忆瓶颈”



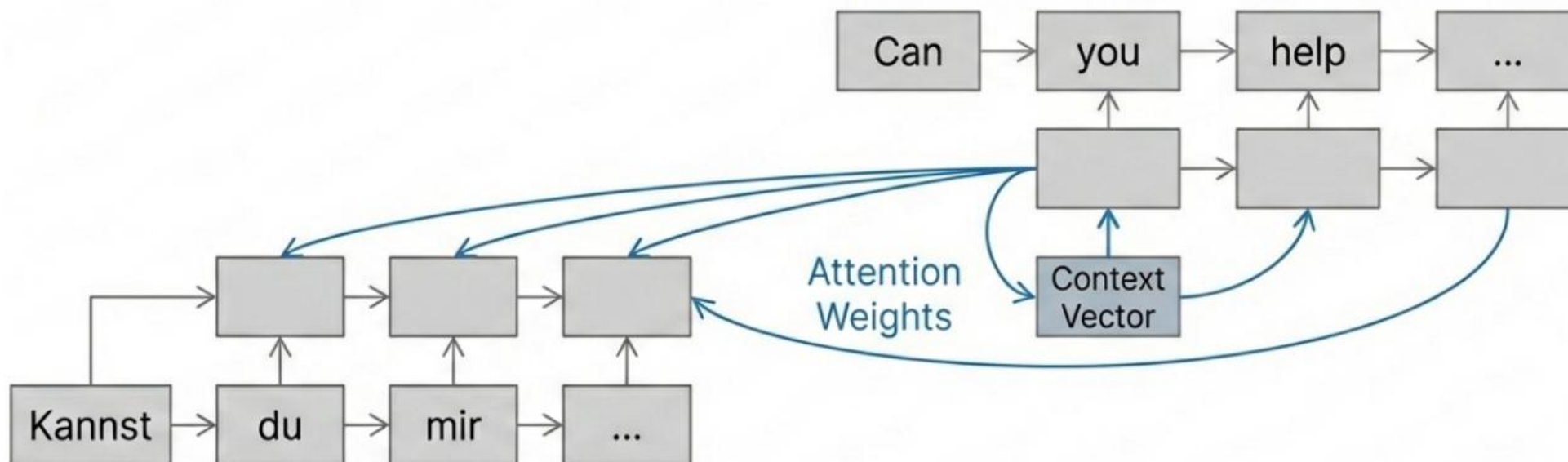
在 Transformer 出现之前，循环神经网络（RNN）以“按顺序逐词”的方式处理文本。用于机器翻译时，编码器必须先把整句输入的语义压缩到一个固定长度的“隐藏状态”（也称上下文向量）中，然后解码器才能开始生成译文。由于这个向量长度是固定的，压缩过程就形成了**信息瓶颈**：句子越长，模型越容易在读到后半段时“忘记”开头的信息，最终导致长句的翻译效果显著变差。

像人类一样分配注意力

注意力机制 Attention Mechanism

注意力机制通过让解码器在生成时访问“全部输入状态的历史信息”（而不仅仅是最后一个隐藏状态），从根本上消除了信息瓶颈。它就像人类翻译员在写译文时，会不时回看原文中的关键单词或短语来确认含义一样。

模型在每一步生成输出词时，都会为不同的输入词动态分配**“重要性权重”**，从而把最相关的信息提取出来辅助当前生成过程。因为可以随用随取，不再需要把整句语义强行压缩进一个固定长度的向量。



自注意力 Self – Attention

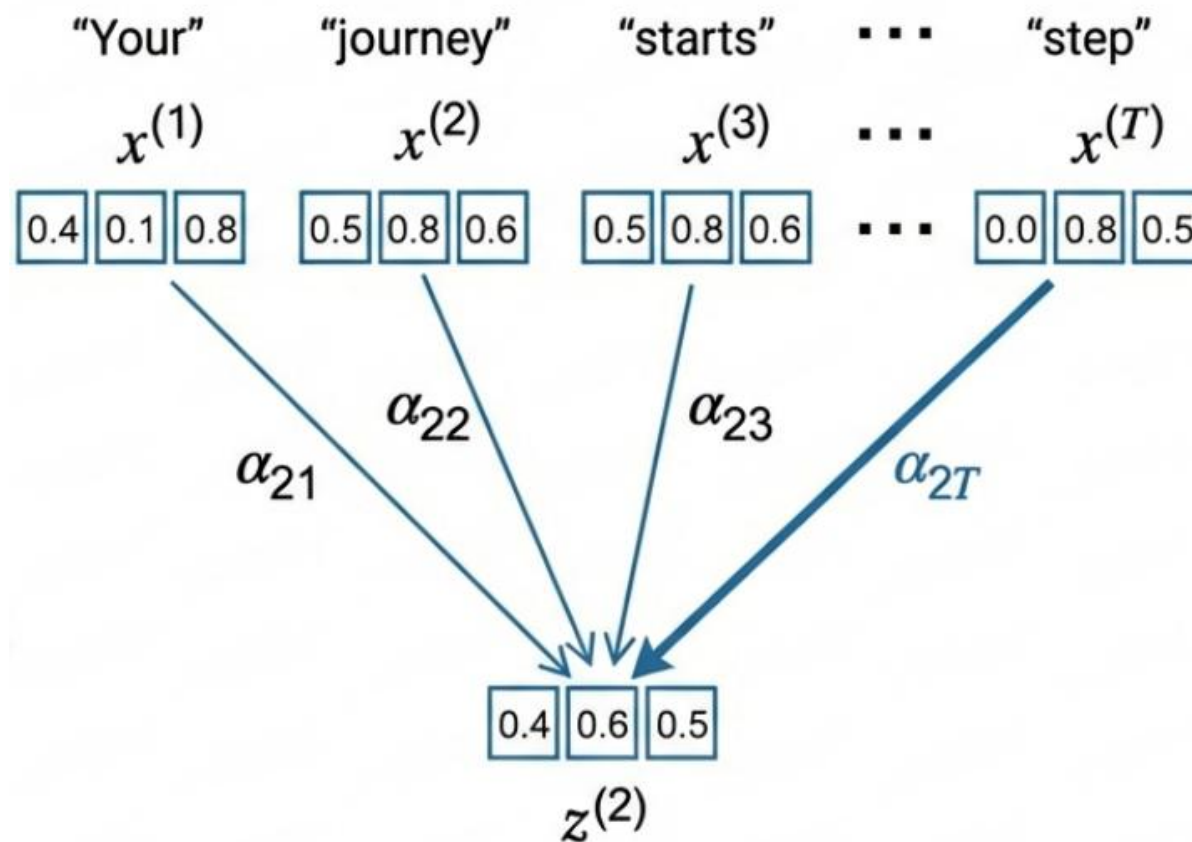
自注意力是在同一条序列内部进行信息交互。也就是说，模型会在一句话里让每个词去“对照”同一句中的其他词，从而理解它在当前语境下的含义。

• 目标

把原始的词向量转换为“具备上下文信息”的词向量
(context-aware embedding)，让每个词不仅代表它本身，还包含它与周围词的关系信息。

• 机制

词语“journey”会去关注同句中的“starts”和“step”等相关词，以确定它在这个具体语境里的含义。最终输出不是简单复制原词向量，而是将该词与其关键邻近词按权重加权融合，得到一个“加权混合”的表示。

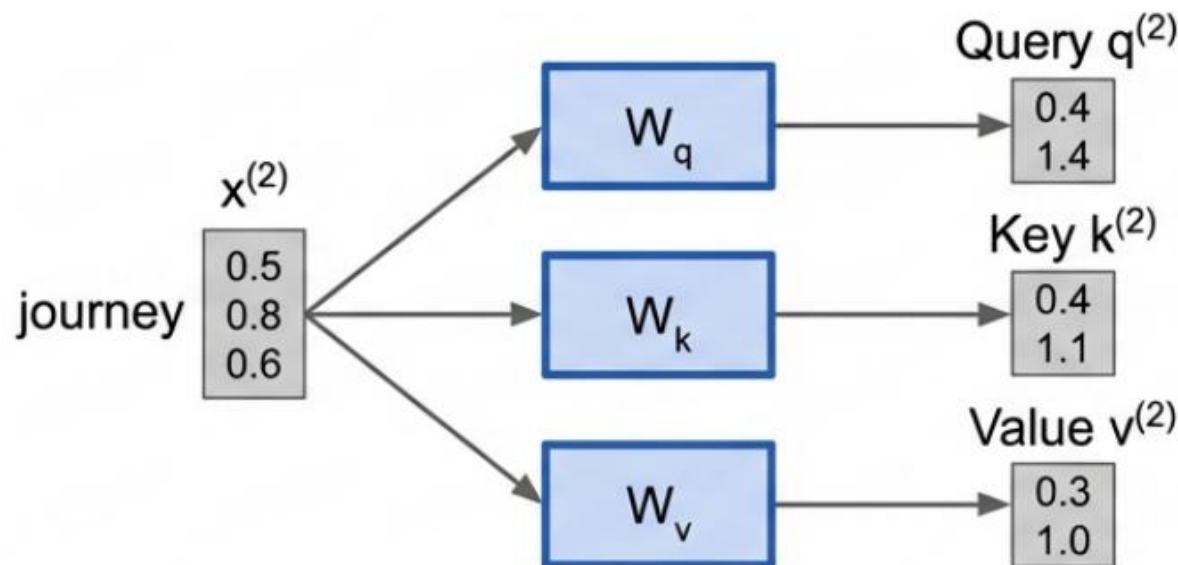


原理：Query、Key 与 Value

核心原理

为了让注意力机制可以被训练，模型会用三组可学习的权重矩阵 (W_q 、 W_k 、 W_v) 把每个输入词投影成三个不同的向量：Query、Key 和 Value。

- **Query (Q)**：表示“这个词想要找什么信息”，也就是它当前的关注需求。
- **Key (K)**：表示“这个词能被别人如何识别”，用于和别人的 Query 做匹配。
- **Value (V)**：则是“这个词真正携带的内容信息”，当匹配成功后会被取出来参与计算。



模型先提出**问题 (Query)**，再去对照**索引 (Key)**找到最相关的条目，最后取回**实际内容 (Value)**用于生成当前的上下文表示。

如何判断相关性：点积 Dot Product



LYCHEE

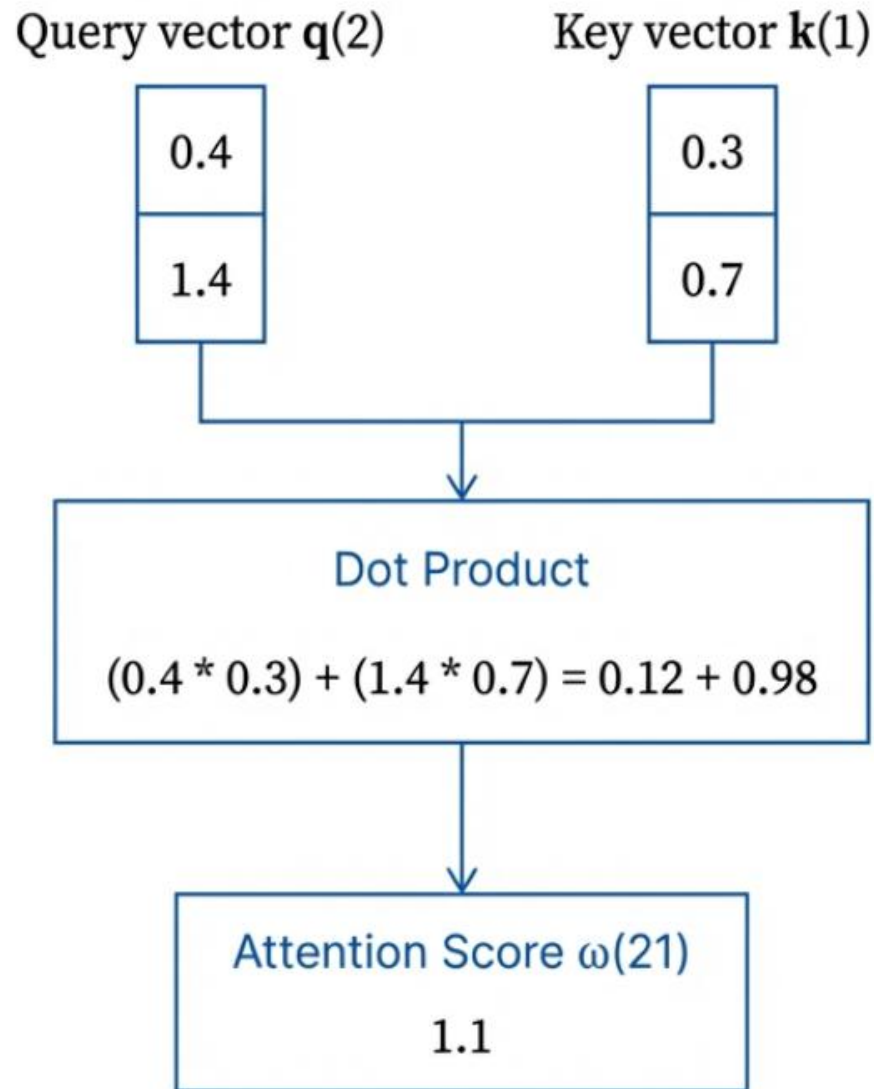


哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

模型要知道哪些词彼此相关，会把“当前词”的 Query (Q) 与“句子中每个词”的 Key (K) 逐一做相似度计算。最常用的方法就是**点积 (Dot Product)**，它是一种衡量两个向量“对齐程度”的数学运算。

- **点积越大**，表示两个向量越对齐，相似度越高，模型就会更“关注”那个词；
- **点积越小**，甚至为负，表示相似度低，模型倾向于忽略该词。

通过这一过程，模型会为句子中“任意两词的组合”产生一个原始的**注意力分数 (attention score)**，也就是后续分配注意力权重的基础。



归一化：Softmax 的作用



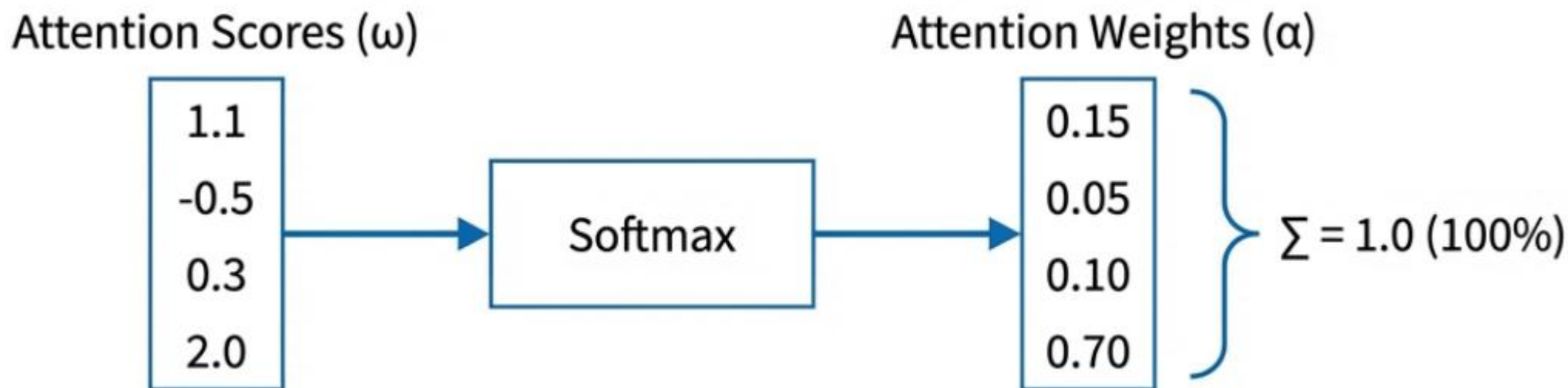
LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

原始的**注意力分数 (attention scores)** 范围可能从负无穷到正无穷，数值跨度很大，既不直观，也不方便直接当作“权重”使用。因此我们会把这些分数输入 **Softmax 函数** 进行归一化，把它们转换成可解释的概率分布。

Softmax 会把所有权重压到 0 到 1 之间，并且保证它们的**总和严格等于 1.0**（也就是 100%）。这会迫使模型在有限的注意力预算里做取舍：把更多权重分配给最相关的词，同时压低不重要信息的权重，从而减少噪声、突出关键线索。



上下文向量：信息的加权融合 Mixing the Information

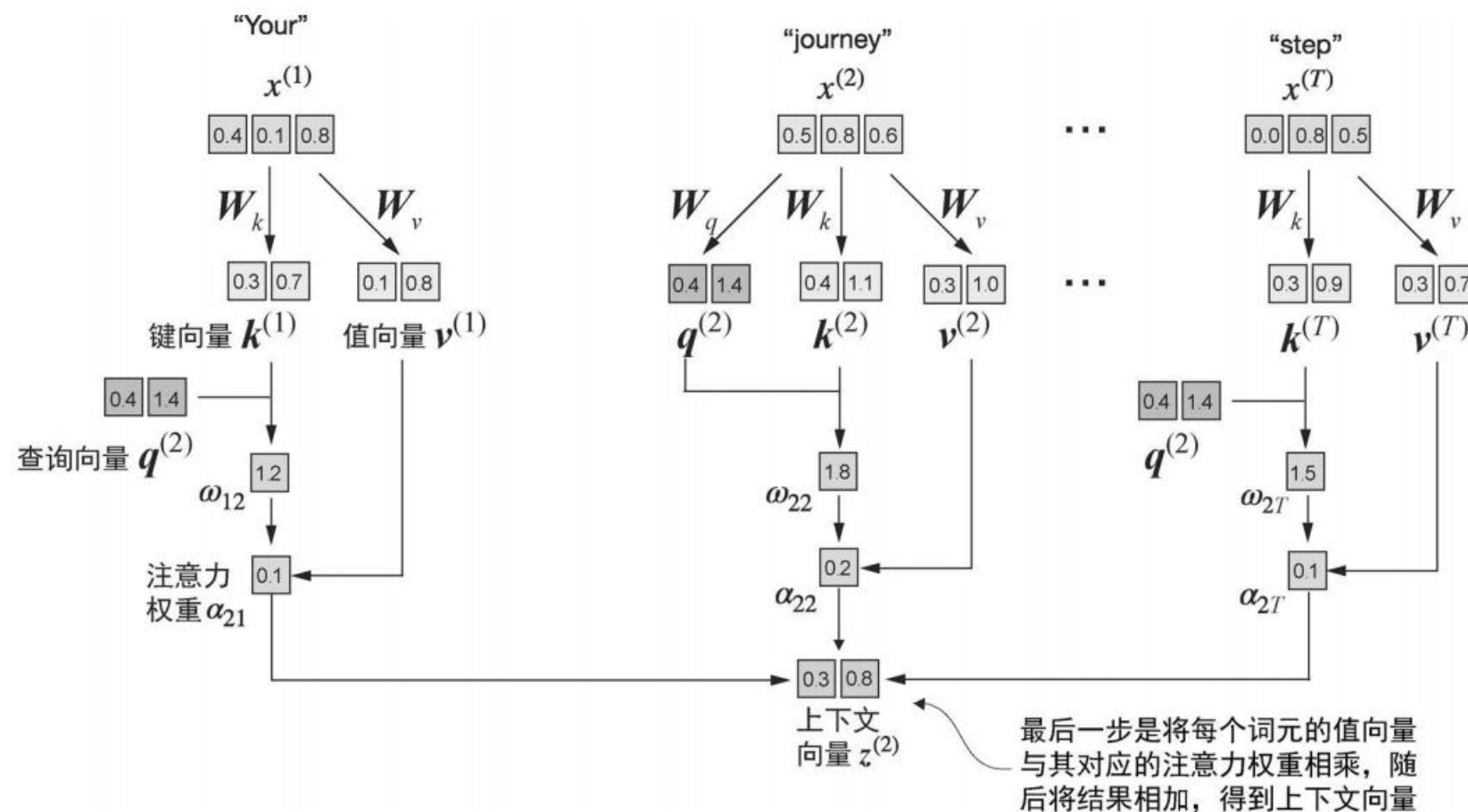
核心逻辑

模型会取出句子中所有词的 Value 向量，并使用注意力权重对它们做加权求和，从而得到当前词的输出表示。

如果某个词与当前词的相关性很高，比如占 90%，那么它的 Value 向量就会以 90% 的比例参与合成；

如果相关性只有 1%，那么它的贡献几乎可以忽略不计。

这样，模型就能把最有用的信息保留下来，把不重要的信息剔除出去。



因果注意力 Causal Attention

标准的自注意力允许一个词查看整句话中的所有词，但在文本生成任务里，这会带来一个关键问题：**模型在预测当前位置的词时，如果能看到后面的“未来词”，就等于提前拿到了答案。**

解决方案：每个位置只能关注它自己以及它之前的词，位于它之后的词必须被**遮住 (mask 掉)**。这样可以防止训练时出现“作弊”，并确保模型只能按照时间顺序逐步生成文本，也就是严格地从左到右、一步一步地写下去。

	Standard Attention				
	The	cat	sat	on	the
The	0.19	0.20	0.14	0.13	0.15
cat	0.20	0.20	0.23	0.20	0.17
sat	0.18	0.20	0.22	0.12	0.17
on	0.16	0.17	0.16	0.18	0.16
the	0.15	0.18	0.14	0.15	0.15

	Causal Attention				
	The	cat	sat	on	the
The	0.19	Masked (Future)			
cat	0.20	0.11			
sat	0.18	0.22	0.13		
on	0.16	0.19	0.14	0.12	
the	0.15	0.17	0.15	0.16	0.15

如何实现 Mask: 负无穷 ($-\infty$) 的逻辑



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

为了在技术上实现“看不见未来”的限制，我们会在 Softmax 之前先处理原始注意力分数，让不该被关注的位置在归一化后自动变成 0。

- (1) 把注意力矩阵右上角三角区域（代表未来位置）的分数全部替换为负无穷 ($-\infty$)
- (2) 在数学上， $e^{-\infty}$ 会趋近于 0，因此这些位置在 Softmax 后几乎不可能分到任何概率质量
- (3) 所有未来词对应的注意力权重会变成严格的 0，从而在计算上“切断连接”，确保模型只能关注当前与过去的信息

Raw Scores					Attention Weights				
0.1	$-\infty$	$-\infty$	$-\infty$	$-\infty$	0.15	0.0	0.0	0.0	0.0
1.2	0.0	$-\infty$	$-\infty$	$-\infty$	0.15	0.22	0.0	0.0	0.0
0.3	0.5	$-\infty$	$-\infty$	$-\infty$	0.15	0.22	0.0	0.0	0.0
0.8	1.3	0.6	$-\infty$	$-\infty$	0.15	0.22	0.18	0.0	0.0
0.5	0.7	0.1	0.1	$-\infty$	0.15	0.24	0.14	0.16	0.0
0.2	0.9	0.5	0.3	1.0	0.15	0.22	0.16	0.15	0.00

多头注意力 Multi-Head Attention

单一的注意力机制往往只能给出一种“理解角度”，容易遗漏语言的复杂关系。为了更全面地建模句子，Transformer 会并行运行多个注意力头，**让模型从不同“视角”同时观察同一段文本。**

比如某个 head 可能**更关注语法结构**，另一个 head 可能**更关注指代与词汇匹配**，还有的 head 会**更关注情绪或语气**。

这些 head 各自输出一份上下文表示，最后会被拼接并再做一次整合，形成更丰富、更立体的表示，从而提升模型对语义、语法与长距离依赖的捕捉能力。

